

Module 1 — ML Fundamentals Practice

6 scenario-based questions, ~5-10 min each. Some have multiple correct answers.

Format on the real test: multiple choice or fill-in-the-blank. Here, write your answer in the cell below each question, then check against the answer cell.

Q1 — L1 vs L2 Regularization

A data scientist is training a linear regression model on a dataset with 500 features. She suspects that only a small number of features are actually predictive. She tries two regularization approaches and observes that one of them produces a model where most feature weights are exactly zero, while the other produces small but nonzero weights for nearly all features.

Which regularization technique produces sparse weights (many exactly zero), and why?

(Also: in what situation would you prefer the other one?)

Your answer: L1 produces many to be exactly zero. The gradient of the objective function (with L1 penalties on the parameters) only depends on the sign of the parameters, not their value. The gradients can therefore push the parameters to zero. For L2 reg, the gradient cannot vanish, and therefore, neither can the params. You would prefer one over the other when it gives better validation loss on held-out data (under the un-augmented objective function).

► Answer

L1 (Lasso) produces sparse weights. The L1 penalty adds $|w|$ to the loss. The subdifferential of $|w|$ at $w=0$ includes 0, so gradient descent can land exactly at zero — the loss surface has a "corner" at zero that acts as an attractor.

L2 (Ridge) adds w^2 to the loss. The gradient is $2w$, which approaches zero as w approaches zero but never produces exact zeros — weights shrink toward zero but stay nonzero.

Prefer L2 when: features are correlated (L1 arbitrarily picks one and zeros the others), or when you believe most features contribute something and want stable small weights rather than hard zeroing.

Q2 — Overfitting

A team trains a decision tree on a customer churn dataset. On the training set, accuracy is 99%. On the held-out test set, accuracy is 61%. A colleague suggests three fixes:

- A) Increase the maximum tree depth
- B) Limit the maximum tree depth
- C) Collect more training data
- D) Remove regularization
- E) Use cross-validation to tune the depth hyperparameter

Which options would likely help? Select all that apply.

Your answer:

► Answer

B, C, E

- B: Limiting depth reduces model complexity, preventing memorization of training noise.
- C: More data makes it harder to overfit — the model must generalize to a larger, more representative set.
- E: Cross-validation lets you find the depth that generalizes, not just the one that fits training data best.

A and D would make overfitting worse.

Q3 — Limitations of Bayes Rule

A hospital uses a Naive Bayes classifier to predict whether a patient has a rare disease (prevalence: 1 in 10,000). The test has 99% sensitivity (true positive rate) and 99% specificity (true negative rate). A patient tests positive.

What is the probability the patient actually has the disease? Is this surprising?
What assumption does Naive Bayes make that may be violated in practice?

```
In [2]: # Compute P(disease | positive test) using Bayes theorem
# P(disease) = 1/10000
# P(positive | disease) = 0.99
# P(positive | no disease) = 0.01

p_disease = 1 / 10000
p_pos_given_disease = 0.99
```

```

p_pos_given_no_disease = 0.01

# implement Bayes theorem here
p_positive = p_pos_given_disease*p_disease \
+ p_pos_given_no_disease*(1 - p_disease)
p_disease_given_pos = ( p_pos_given_disease * p_disease ) / p_positive

print(f"P(disease | positive) = {p_disease_given_pos:.4f}")

```

P(disease | positive) = 0.0098

Your written answer (what assumption does Naive Bayes make?):

► Answer

```

p_positive = p_pos_given_disease * p_disease +
p_pos_given_no_disease * (1 - p_disease)
p_disease_given_pos = (p_pos_given_disease * p_disease) / p_positive
# ≈ 0.0098, about 1%

```

Even with a 99% accurate test, the positive predictive value is only ~1% because the disease is so rare. Most positives are false positives.

Naive Bayes assumption: features are conditionally independent given the class label. In practice, symptoms are often correlated (e.g., fever and chills co-occur), violating this assumption.

Q4 — Choosing k in kNN

A data scientist is tuning k for a kNN classifier on a binary classification task. She observes:

- k=1: training accuracy 100%, test accuracy 72%
- k=50: training accuracy 81%, test accuracy 80%
- k=200: training accuracy 74%, test accuracy 74%

What is happening at k=1 vs k=200? What is the best choice here and how would you find the optimal k in practice?

Your answer:

► Answer

- **k=1:** Each point is its own nearest neighbor → perfect training accuracy, but high variance/overfitting. Decision boundary is jagged.
- **k=200:** Very smooth decision boundary, but may underfit if the neighborhood is so large it spans class boundaries. High bias.
- **k=50:** Best test accuracy here — good bias-variance tradeoff.

In practice: use cross-validation (e.g., 5-fold CV) to sweep k values and pick the one with best validation accuracy. Also prefer odd k for binary classification to avoid ties.

Q5 — GBM vs Random Forest

A team is comparing Gradient Boosting (GBM) and Random Forest on a tabular dataset. They observe that GBM achieves lower error but takes much longer to train and is more sensitive to hyperparameters.

Explain why GBM tends to outperform Random Forest but is harder to tune. When would you prefer Random Forest?

(Also: what does each method do to reduce variance/bias?)

Your answer:

► Answer

Random Forest: bagging (bootstrap aggregating) — trains trees in parallel on random subsets of data and features, then averages. Reduces variance. Trees are decorrelated by feature subsampling. Fast, robust, hard to overfit badly.

GBM: boosting — trains trees sequentially, each correcting the residuals of the previous. Reduces bias as well as variance. More powerful but: learning rate, tree depth, and number of estimators all interact, requiring careful tuning. Can overfit if not regularized.

Prefer Random Forest when: you need fast training, robust defaults, or interpretability matters more than squeezing out last % of accuracy.

Q6 — Neural Network Forward Pass (Computation)

Given the network below, compute the output. No libraries — just arithmetic.

Architecture:

- Inputs: $x_1=4$, $x_2=2$, $x_3=7$
- Hidden layer (f_1 , f_2): linear activation, bias=0
 - $f_1 = 0.5x_1 + 0.3x_2 + 0.1x_3$ (weights: 0.5 from x_1 , 0.3 from x_2 ... wait — use the diagram weights)
 - Weights to f_1 : $x_1 \rightarrow 0.2$, $x_2 \rightarrow 0.3$, $x_3 \rightarrow 0.4$
 - Weights to f_2 : $x_1 \rightarrow 0.5$, $x_2 \rightarrow 0.1$, $x_3 \rightarrow 0.5$
- Output layer (f_3): sigmoid activation, bias=0

- Weights: $f1 \rightarrow 0.2$, $f2 \rightarrow 0.3$

Compute the output, rounded to 3 decimal places.

```
In [3]: import math

x1, x2, x3 = 4, 2, 7

# Hidden layer (linear)
f1 = 0.2*x1 + 0.3*x2 + 0.4*x3
f2 = 0.5*x1 + 0.1*x2 + 0.5*x3

# Output layer (sigmoid)
z = f1*0.2 + f2*0.3
output = 1 / (1 + math.exp(-z))

print(f"f1={f1}, f2={f2}, z={z}, output={output:.3f}")
```

f1=4.2, f2=5.7, z=2.55, output=0.928

► Answer

```
f1 = 0.2*4 + 0.3*2 + 0.4*7 # = 0.8 + 0.6 + 2.8 = 4.2
f2 = 0.5*4 + 0.1*2 + 0.5*7 # = 2.0 + 0.2 + 3.5 = 5.7
z  = 0.2*f1 + 0.3*f2       # = 0.84 + 1.71 = 2.55
output = 1/(1+exp(-2.55))  # ≈ 0.928
```